

Practical-8**Task -1:**

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    printf("Dynamic Memory Allocation Demonstration\n\n");

    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        a[i] = (i + 1) * 10;

    printf("malloc values: ");

    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);

    printf("\n");

    int *b = (int *)calloc(5, sizeof(int));
```

```
if (b == NULL)
{
    printf("calloc failed\n");
    free(a);
    return 1;
}
printf("calloc values: ");
for (i = 0; i < 5; i++)
    printf("%d ", b[i]);
printf("\n");
int *temp = (int *)realloc(a, 10 * sizeof(int));
if (temp == NULL)
{
    printf("realloc failed\n");
    free(a);
    free(b);
    return 1;
}
a = temp;
for (i = 5; i < 10; i++)
    a[i] = (i + 1) * 10;
printf("After realloc: ");
for (i = 0; i < 10; i++)
    printf("%d ", a[i]);
printf("\n");
free(a);
```

```
free(b);

printf("\nAll allocated memory has been freed.\n");

return 0;
}
```

Output:

```
root@Rahul: ~
root@Rahul:~# nano s8task1.c
root@Rahul:~# gcc s8task1.c -o s8task1
root@Rahul:~# ./s8task1
Dynamic Memory Allocation Demonstration

malloc values: 10 20 30 40 50
calloc values: 0 0 0 0 0
After realloc: 10 20 30 40 50 60 70 80 90 100

All allocated memory has been freed.
root@Rahul:~# |
```

Task -2:

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (50 * 1024 * 1024)
```

```
int main()
{
    char *data;
    pid_t pid;
    data = malloc(SIZE);
    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }
    for (size_t i = 0; i < SIZE; i++)
        data[i] = 'A';
    printf("Parent PID: %d\n", getpid());
    printf("Allocated: 50 MB\n");
    printf("\nBefore fork()\n");
    printf("Press Enter to continue...");
    getchar();
    pid = fork();
    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }
    if (pid == 0)
    {
```

```
printf("\nChild PID: %d\n", getpid());
printf("Parent PID: %d\n", getppid());
printf("Child has started.\n");
printf("Press Enter before modifying memory...");
getchar();
for (size_t i = 0; i < SIZE; i += 4096)
    data[i] = 'C';
printf("Child modified the allocated memory.\n");
printf("Press Enter before child exits...");
getchar();
free(data);
exit(0);
}
else
{
    printf("\nParent after fork()\n");
    printf("Child PID: %d\n", pid);
    printf("Parent has not modified the memory.\n");
    printf("Press Enter to wait for child...");
    getchar();
    wait(NULL);
    printf("Child finished.\n");
}
free(data);
return 0;
}
```

Output:

```
root@Rahul: ~  
root@Rahul:~# nano task2.c  
root@Rahul:~# gcc -g -O0 task2.c -o task2  
root@Rahul:~# ./task2  
Parent PID: 584  
Allocated: 50 MB  
  
Before fork()  
Press Enter to continue...  
  
Parent after fork()  
Child PID: 608  
Parent has not modified the memory.  
Press Enter to wait for child...  
Child PID: 608  
Parent PID: 584  
Child has started.  
Press Enter before modifying memory...
```

```
root@Rahul:~# ps -o pid,ppid,rss,vsz,stat,cmd -p 584  
PID   PPID  RSS   VSZ  STAT CMD  
584    563 52816 53896 S+   ./task2  
root@Rahul:~# ps -o pid,ppid,rss,vsz,stat,cmd -p 584,608  
PID   PPID  RSS   VSZ  STAT CMD  
584    563 52944 53896 S+   ./task2  
608    584 52080 53896 S+   ./task2  
root@Rahul:~#
```